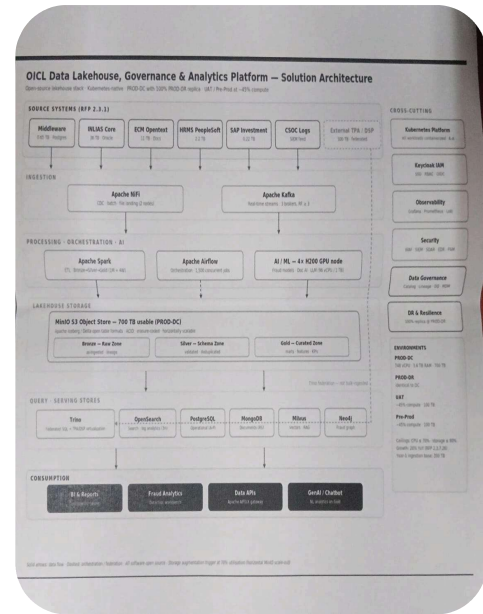




Explain me this architecture in detail in simpler words

Worked for 8s >

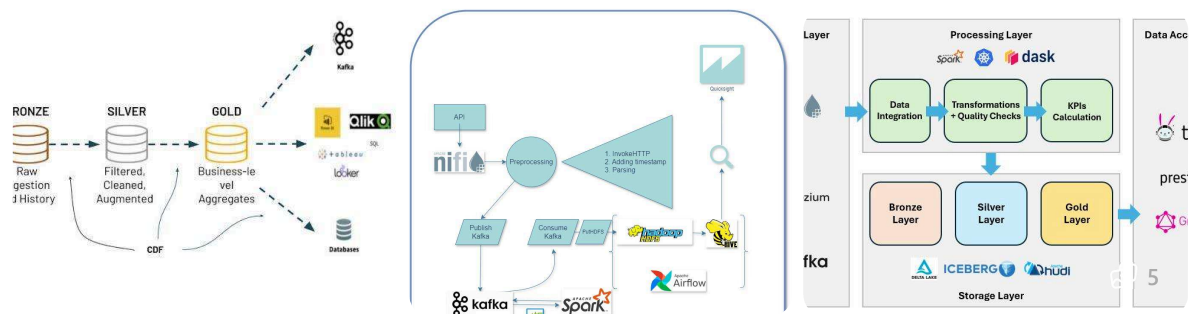
Absolutely. Think of this architecture as a **big data factory for OICL**.

The basic idea is:

Data comes from many OICL systems → gets collected → cleaned and processed → stored in a Data Lakehouse → made available for search/analytics/AI → finally consumed by users and applications.

The diagram is divided into several layers. I'll explain each layer in very simple terms and then explain how data actually flows from left/top to the final applications.

1. Overall picture



You can understand the architecture like this:

OICL Source Systems





And on the side, there are **common services** that support the entire architecture:

Kubernetes
Keycloak IAM
Observability
Security
Data Governance
DR & Resilience

These are not just one layer; they support the whole platform.

2. SOURCE SYSTEMS – Where does the data come from?

At the top you have:

SOURCE SYSTEMS

These are the existing OICL systems that generate business data.

The architecture shows:

Source System	Simple meaning
Middleware	Data/integration coming through middleware systems
INLIAS Core	Core insurance application/system
ECM Opentext	Enterprise content/document management
HRMS PeopleSoft	Employee/HR-related information
SAP Investment	Investment/financial information
CSOC Logs	Cybersecurity/security operation logs
External TPA / DSP	External third-party data providers

So imagine OICL has information sitting in **7 different places**.

For example:

```
INLIAS → Policy data
SAP → Investment data
PeopleSoft → Employee data
ECM → Documents
CSOC → Security logs
TPA → External insurance/claim data
```

The problem is that these systems are separate.

The Data Lakehouse platform brings this information together.

3. INGESTION – How does data enter the platform?

This is the next layer.

There are two important technologies here:

Apache NiFi

and

Apache Kafka

3.1 Apache NiFi

Think of **NiFi** as a **data transportation system**.

It takes data from source systems and moves it into the platform.

For example:

```
INLIAS
  ↓
  ↓
Apache NiFi
  ↓
Data Lakehouse
```

NiFi can handle:

- Batch data
- File transfers
- Database data
- CDC
- Data routing

- Data transformation
- Data movement

The architecture specifically mentions:

CDC, batch, file landing

What is CDC?

CDC = **Change Data Capture**.

Suppose the INLIAS database has:

Policy 101 → Premium = ₹10,000

Tomorrow premium changes to:

₹12,000

Instead of copying the entire database again, CDC can capture:

"Policy 101 changed from ₹10,000 to ₹12,000."

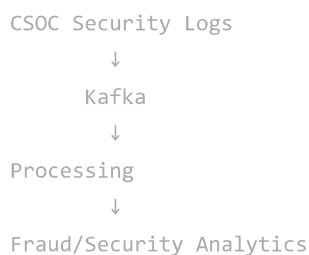
This saves time and resources.

4. Apache Kafka – For real-time data

Kafka is used when data needs to move **very quickly / in real time**.

Think of Kafka as a **high-speed data highway**.

For example:



The architecture mentions:

Real-time streams

So:

NiFi

is more suitable for:

- Batch

- Files
- CDC
- Data movement

Kafka

is more suitable for:

- Real-time streams
- Events
- Continuous data

5. PROCESSING – Now we clean and transform the data

Once data enters the platform, raw data is usually **not ready for business users**.

It may contain:

- Duplicate records
- Wrong formats
- Missing values
- Different date formats
- Different naming conventions
- Invalid records

That's where **Apache Spark** comes in.

6. Apache Spark – The data processing engine

Think of Spark as the **data cleaning and processing machine**.

For example, suppose INLIAS gives:

```
Customer Name = RAKSHITHA
Date = 11/08/2026
Premium = "10,000"
```

Another system gives:

```
Customer_Name = Rakshitha
Date = 2026-08-11
Premium = 10000
```

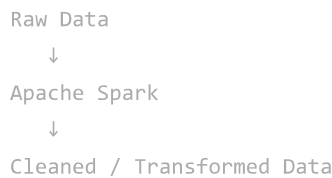
Spark can standardize this information.

It can perform:

- Data cleaning

- Data transformation
- Aggregation
- Joins
- ETL/ELT
- Large-scale processing
- Feature preparation for AI/ML

So:



7. Apache Airflow – The manager/coordinator

This is an important distinction.

Airflow does not primarily process the data.

Instead, it **orchestrates the jobs**.

Think of Airflow as the **project manager**.

For example:



Airflow can control this sequence.

So:

Tool	Simple role
NiFi	Moves data
Kafka	Moves real-time events

Tool	Simple role
Spark	Processes data
Airflow	Schedules/orchestrates jobs

8. AI / ML – 4 × H200 GPU Nodes

The architecture also has:

AI / ML – 4 × H200 GPU node

This is the **AI/ML computing infrastructure**.

GPU = Graphics Processing Unit.

But GPUs are extremely useful for AI because they can perform huge numbers of mathematical calculations in parallel.

These GPUs can support things such as:

- Fraud detection
- Machine learning
- Deep learning
- LLM workloads
- GenAI
- Model training
- Model inference

The diagram mentions:

Fraud models, Doc AI, LLM

So the platform isn't only a traditional data warehouse.

It is also intended to support **AI/ML and GenAI workloads**.

9. LAKEHOUSE STORAGE – The heart of the architecture

Now we reach the most important part:

MinIO S3 Object Store – 700 TB usable (PROD-DC)

This is essentially the **main storage area** of the platform.

Think of MinIO as a giant digital warehouse.

It stores the organization's data.

The architecture uses:

S3-compatible object storage

MinIO provides object storage similar to Amazon S3, but it can be deployed in OICL's own data center.

10. Why is it called a Data Lakehouse?

A traditional **Data Lake** stores huge amounts of raw data.

A traditional **Data Warehouse** stores highly structured, cleaned data for reporting.

A **Lakehouse** tries to combine both approaches.

So OICL can store:

Raw data
Structured data
Semi-structured data
Documents
Logs
AI datasets
Analytics datasets

in one platform.

11. Bronze – Silver – Gold

This is extremely important for understanding the architecture.

Inside MinIO you have three zones:



Bronze = Raw data

Think:

"Whatever came from the source."

Example:

SAP
↓
Bronze

The data is stored almost as received.

It may contain:

- Duplicate records
- Different formats
- Missing values
- Original source structure

Bronze is useful because you **don't lose the original data**.

12. Silver = Cleaned data

Silver is where the data becomes standardized.

Example:

Bronze
↓
Spark
↓
Silver

Data is:

- Cleaned
- Validated
- Deduplicated
- Standardized
- Schema-aligned

For example:

Bronze:

"11/08/26"
"2026-08-11"
"11-Aug-2026"

↓ Spark

Silver:

2026-08-11

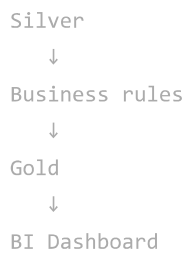
13. Gold = Business-ready data

Gold is the most refined layer.

This is where data is prepared for:

- Reports
- Dashboards
- Analytics
- Fraud detection
- APIs
- AI

For example:



So remember:

Bronze

RAW

Silver

CLEANED

Gold

BUSINESS READY

This is one of the most important concepts in this architecture.

14. QUERY / SERVING STORES

Now comes another important part.

The data is stored in MinIO, but different applications may need to access it in different ways.

That's why the architecture has several **query/serving technologies**.

15. Trino

Trino is the **query engine**.

Think of it as:

"I want to ask questions about the data."

For example:

```
SELECT *  
FROM policy_data  
  
SQL
```

Trino can query large datasets in the lakehouse.

It can also provide a **federated query** capability.

Meaning it can potentially query data from multiple sources without physically moving everything into one database.

16. OpenSearch

OpenSearch is mainly useful for:

- Search
- Logs
- Log analytics
- Text search
- Operational analytics

For example:

```
Search all security logs  
Search policy documents  
Search transaction events
```

It is particularly useful when users need **fast search over large amounts of data**.

17. PostgreSQL

PostgreSQL is a traditional relational database.

It is useful for:

- Structured operational data
- Metadata
- Configuration

- Application data
- Relational queries

Think:

Tables + rows + columns + SQL.

18. MongoDB

MongoDB is a **document database**.

Instead of everything being rigidly structured into tables, it can store JSON-like documents.

Useful for:

- Semi-structured data
- Application data
- Flexible schemas
- Documents

19. Milvus

This is particularly interesting because it is associated with **AI/GenAI**.

Milvus is a **vector database**.

Imagine you have:

```
Insurance document
↓
AI converts it into numbers/vectors
↓
Milvus stores those vectors
```

Then you can perform **semantic search**.

For example, a user asks:

"Show me policies related to fire damage."

The system doesn't necessarily need exact keyword matching.

It can find documents with **similar meaning**.

This is very useful for:

- RAG

- GenAI
- Semantic search
- Document search
- AI assistants

20. Neo4j

Neo4j is a **graph database**.

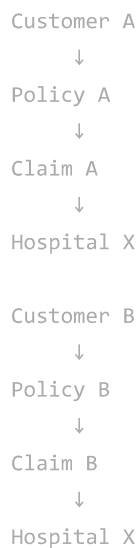
This is useful when relationships are important.

For example:



Fraud detection can benefit greatly from this.

For example:



The graph can identify suspicious relationships.

That's why Neo4j is particularly useful for **fraud analytics / relationship analysis**.

21. Why so many databases?

This is a common question.

You might wonder:

"Why not just use PostgreSQL?"

Because different workloads require different technologies.

Technology	Best suited for
MinIO	Large-scale data storage
Trino	Querying lakehouse data
OpenSearch	Search & log analytics
PostgreSQL	Relational data
MongoDB	Document/semi-structured data
Milvus	Vector/semantic search
Neo4j	Relationships/graphs

So this is a **polyglot architecture**.

Different tools are used for different workloads.

22. CONSUMPTION LAYER

This is where the actual users see the value.

The architecture shows four major consumption areas.

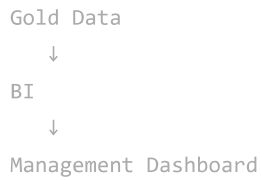
22.1 BI & Reports

BI = Business Intelligence.

This is for:

- Dashboards
- Management reports
- KPIs
- MIS
- Operational reports
- Business analysis

For example:



23. Fraud Analytics

This is for detecting potentially fraudulent activity.

For example:



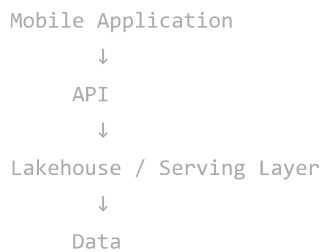
The AI/ML infrastructure + Spark + Neo4j + lakehouse data can all contribute here.

24. Data APIs

Applications may not directly access the lakehouse.

Instead, they can access data through APIs.

For example:



The architecture shows:

Apache APISIX gateway

This acts as an API gateway.

25. GenAI / Chatbot

This is the AI-facing consumption layer.

Imagine an OICL employee asks:

"Show me the claims related to motor insurance in Karnataka during the last six months."

The system can potentially:



This is where the **Milvus + AI/ML + data lakehouse** combination becomes important.

26. CROSS-CUTTING SERVICES

Now look at the right side.

These are called **cross-cutting services** because they support **multiple or all layers**.

They are not just another processing stage.

27. Kubernetes

The architecture is described as:

Kubernetes-native

Kubernetes manages the applications/services running on the platform.

Think of Kubernetes as the **overall infrastructure manager for software containers**.

For example:


```
Kubernetes
├─ NiFi
├─ Kafka
├─ Spark
├─ Airflow
├─ Trino
├─ APIs
└─ Other services
```

It helps with:

- Deployment
- Scaling
- High availability
- Container management
- Resource management
- Service orchestration

28. Keycloak IAM

IAM = Identity and Access Management.

Keycloak manages:

Who are you and what are you allowed to access?

For example:

```
User: Rakshitha
    ↓
Login
    ↓
Keycloak
    ↓
Authentication
    ↓
Role = Data Analyst
    ↓
Access permitted resources
```

It can support:

- SSO
- Authentication
- Authorization
- RBAC
- Identity federation

29. Observability

The architecture mentions:

Grafana, Prometheus, Loki

These help the administrators understand whether the platform is working correctly.

Think:

Prometheus

"How is the system performing?"

Example:

```
CPU = 70%  
RAM = 80%  
Kafka lag = ...
```

Grafana

"Show me this information visually."

It provides dashboards.

Loki

"What's happening in the logs?"

So:

```
Prometheus → Metrics  
Grafana → Dashboards  
Loki → Logs
```

30. Security

The architecture mentions security components such as:

- WAF
- SIEM
- SOAR
- EDR
- IAM

The exact implementation would depend on the final OEM/product selection.

But conceptually:

WAF

Protects web applications.

SIEM

Collects and analyzes security events.

SOAR

Automates security response.

EDR

Monitors endpoints.

IAM

Controls identities/access.

So security surrounds the entire platform.

31. Data Governance

This is another extremely important area.

The architecture mentions:

Catalog, Lineage, DQ, MDM

Let's simplify.

Data Catalog

"What data do we have?"

Example:

```
graph TD
    A[Customer Table]
    B[Policy Table]
    C[Claims Table]
    D[Investment Table]
```

Data Lineage

"Where did this data come from?"

For example:

```
graph TD
    A[SAP] --> B[NiFi]
```



You can trace the journey.

DQ = Data Quality

"Is the data correct?"

Checks might include:

- Missing values
- Duplicate records
- Invalid values
- Incorrect formats
- Referential integrity

MDM = Master Data Management

"Which version of the customer/business entity is the master?"

For example, if three systems have slightly different customer names/details, MDM helps establish a consistent master record.

32. DR & Resilience

DR = Disaster Recovery.

The architecture says:

100% replica @ PROD-DR

That means there is another environment intended to provide a replica/copy of the production setup/data for disaster recovery.

Think:



Replication



PROD-DR

If the primary data center has a major failure, the DR environment can be used to restore/continue services according to the defined DR design and RPO/RTO.

33. Environments

The right side also shows:

PROD-DC

Production environment at the primary data center.

This is the **live system**.

PROD-DR

Production disaster recovery environment.

This is the backup/recovery site.

UAT

User Acceptance Testing

Users test whether the system works as expected before production deployment.

Pre-Prod

Pre-Production

Environment that is very similar to production and used for final testing before going live.

So:

Development/Test



UAT



Pre-Prod



PROD



DR

The exact deployment lifecycle can vary, but that's the simple concept.

34. What does "UAT / Pre-Prod at -45% compute" mean?

This is essentially a **sizing/capacity statement**.

It means the non-production environments are planned with reduced compute capacity compared with production.

For example, if production has:

100 compute units

then a 45% reduction would mean approximately:

55 compute units

for that environment, subject to the actual RFP sizing and architecture assumptions.

This is a **cost optimization approach** because UAT and Pre-Prod generally don't need the same processing capacity as the live production environment.

35. The complete data journey

Now let's put everything together.

Suppose OICL receives a new claim.

Step 1 – Source

The claim exists in an OICL source application.

INLIAS

↓

Step 2 – Ingestion

NiFi captures the data.

INLIAS

↓

NiFi

If it is a real-time event, Kafka may be involved.

INLIAS / Events

↓

Kafka

↓

Step 3 – Processing

Spark cleans/transforms the data.

NiFi/Kafka

↓

Spark

↓

Step 4 – Raw storage

Original data goes into:

MinIO

↓

Bronze

↓

Step 5 – Clean data

Spark processes it:

Bronze

↓

Spark

↓

Silver

↓

Step 6 – Business-ready data

Business transformations happen:

Silver

↓

Business rules

↓

Gold

↓

Step 7 – Serving

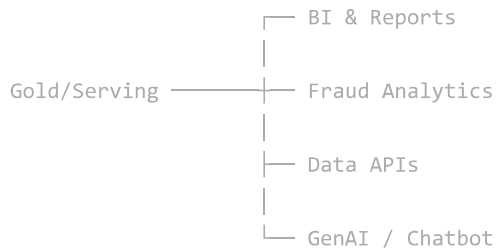
Depending on the requirement:



↓

Step 8 – Consumption

Finally:

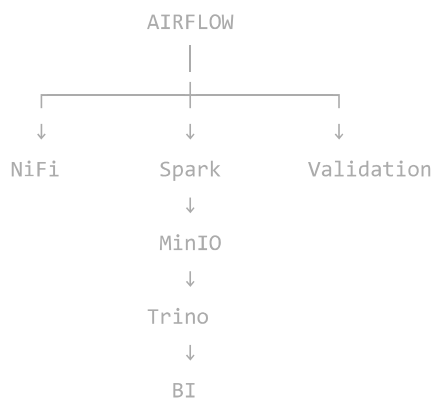


36. Where does Airflow fit?

A common misconception is that Airflow sits in the data path.

It is better to think of it as the **orchestrator controlling the workflow**.

For example:



Airflow can tell each job:

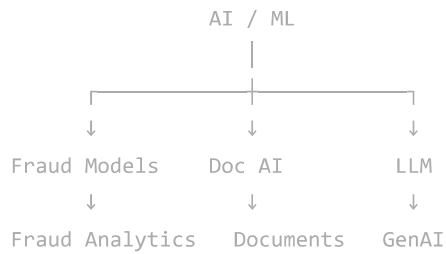
"Run this first, then this, then this."

37. Where does AI fit?

AI is not necessarily a separate isolated layer.

It can interact with several layers.

For example:



AI can consume data from the lakehouse and produce predictions/models that are then used by applications.

38. Why this architecture is called a "Data Lakehouse, Governance & Analytics Platform"

Because it is actually combining **three big capabilities**.

1. Data Lakehouse

MinIO
+
Bronze/Silver/Gold
+
Spark
+
Trino

Stores and processes organizational data.

2. Governance

Catalog
Lineage
Data Quality
MDM
Security
IAM

Controls and manages the data.

3. Analytics/AI

BI
Fraud Analytics
AI/ML

Uses the data to generate business value.

39. One very simple real-life analogy

Imagine OICL's entire data platform is a **large supermarket**.

Source systems = suppliers

SAP
INL IAS
HRMS
ECM
CSOC
TPA

They bring products/data.

NiFi/Kafka = delivery trucks

They bring the data into the supermarket.

Spark = cleaning/processing team

They clean and organize the products.

MinIO = warehouse

Stores everything.

Bronze = products just received

Raw.

Silver = cleaned and organized products

Standardized.

Gold = neatly packed products ready for customers

Business-ready.

Trino/OpenSearch/PostgreSQL/MongoDB/Milvus/Neo4j = different counters

Each specializes in serving a particular type of customer/request.

BI/Fraud/API/GenAI = customers

They consume the data.

Kubernetes = supermarket manager

Keeps the whole operation running.

Keycloak = security guard

Controls who can enter/access what.

Governance = inventory manager

Knows what is stored, where it came from and whether it is good quality.

DR = second supermarket

If the main supermarket becomes unavailable, the second location can support recovery.

40. The most important thing to remember

If you have to explain this architecture in a meeting, don't try to explain every product first.

Explain it in **6 simple stages**:

Stage	Components	Simple explanation
1. Source	INLIAS, SAP, HRMS, ECM, CSOC, TPA	Where OICL's data originates
2. Ingestion	NiFi, Kafka	Bring data into the platform
3. Processing	Spark, Airflow, AI/ML	Clean, transform, orchestrate and analyze data
4. Storage	MinIO + Bronze/Silver/Gold	Store raw → cleaned → business-ready data
5. Serving	Trino, OpenSearch, PostgreSQL, MongoDB, Milvus, Neo4j	Make data available for different types of queries/applications
6. Consumption	BI, Fraud, APIs, GenAI	Where users/applications actually use the data

And across **all six stages**:

- Kubernetes
- IAM
- Security
- Observability
- Data Governance
- DR

provide the supporting foundation.

★ One-line explanation for your OICL RFP meeting

You can say:

"The proposed OICL Data Lakehouse architecture consolidates data from multiple source systems through NiFi and Kafka, processes and orchestrates it using Spark and Airflow, stores it in a MinIO-based S3 lakehouse using Bronze-Silver-Gold zones, exposes it through specialized query and serving engines such as Trino, OpenSearch, PostgreSQL, MongoDB, Milvus and Neo4j, and finally delivers BI, fraud analytics, APIs and GenAI capabilities, with Kubernetes, IAM, security, observability, governance and DR providing the common platform services."

That is essentially the **whole diagram in one sentence**.



ChatGPT can make mistakes. Check important info.